



Linear Algebra with Computer Science

T-201-LINC

EXAM

Teacher: Christoph Lohrmann and Gabriel Jouan

Date: April 22nd, 2024

Time: 9:00 am - 12:00 pm

Name: _____

ID Number (Kennitala): _____

See next page for further instructions.

Instructions

Schedule for the exam: The exam starts at 9:00 am. You have until 12:00 pm to solve the problems of the exam. The exam is in three parts:

- Part 1 Calculations. This part has 6 problems, but the **best 5 count**. Each is worth 7%.
- Part 2: Coding and Algorithms. 4 problems, but the **best 3 count**. Each is worth 15%.
- Part 3: Theory. This part has 3 problems, but the **best 2 counts**. Each is worth 10%.

Permitted exam materials:

- A single sheet of A4 paper. You can write on the front and the back. The paper is (1) handwritten by pen and/or (2) digitally handwritten and printed out. (3) Text typed e.g., via keyboard is also possible but with a minimum font size 10 with font Times New Roman / Arial (or another font of roughly the same size with font size 10). First and last name needs to be at the top of the paper. The paper may not contain printed out lecture slides (or screenshots of slides and other content found online or from digital sources - this includes screenshots of proofs, figures and graphs).
- A calculator. Not a laptop, a tablet, a smartphone, or a device that connects to the internet. The calculator may not be a scientific or a graphical calculator.

You are not allowed to communicate with each other or get external help. Your solutions must be your own.

You should always show your work.

- If you use a formula make sure to write it in your solution, even though you use a calculator to calculate the value.
- Explain and motivate your work.
- When you need to write code in your solutions, we will not deduct points for simple syntax errors, and pseudo-code is acceptable. Note, however, that the grade for questions with code is determined both by correctness and running time.

Good luck!

Part 1

1. (a) (4%) Consider the linear system given below. Write it as a matrix times a vector equation.

$$5x_1 + 2x_2 - x_3 = 1$$

$$3x_2 + 2x_3 = -3$$

$$2.5x_3 = 7.5$$

Show work.

☐ Need more space? Continued here: _____

Solution:

$$\begin{pmatrix} 5 & 2 & -1 \\ 0 & 3 & 2 \\ 0 & 0 & 2.5 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 1 \\ -3 \\ 7.5 \end{pmatrix}$$

- (b) (3%) Solve the system, in any way you want.

Show work.

☐ Need more space? Continued here: _____

Solution: With backwards substitution we get the following sequence of equations

$$2.5x_3 = 7.5$$

$$x_3 = 3$$

$$3x_2 + 2x_3 = -3$$

$$3x_2 + 2 \cdot 3 = -3$$

$$3x_2 = -9$$

$$x_2 = -3$$

$$5x_1 + 2x_2 - x_3 = 1$$

$$5x_1 + 2 \cdot (-3) - 3 = 1$$

$$5x_1 = 10$$

$$x_1 = 2$$

2. (a) (3%) There are two matrices A and B and you know that $B \cdot A$ is the identity matrix. Can one conclude that these matrices are inverses of each other? If yes, why? If no, why not and what else would have to be known?

Show work.

☐ Need more space? Continued here: _____

Solution: It is not always true that A and B are inverses if $B \cdot A$ is the identity matrix, so we cannot conclude that they are inverses just based on the information above.

We could only conclude that A and B are inverses if both $B \cdot A$ and $A \cdot B$ result in the identity matrix OR if A and B are square matrices and $B \cdot A$ is the identity matrix. Neither information is provided in the task

Thus, at least one out of two aspects would need to be known additionally:

1. Whether also $A \cdot B$ is the identity matrix
2. Whether A and B are square matrices

If one of these is known and true, then A and B are inverses of each other.

- (b) (4%) Below are two matrices, A and B .

$$A = \begin{pmatrix} 7 & 2 & 0 \\ 1 & 2 & 3 \\ 2 & -3 & -4 \end{pmatrix} \quad B = \begin{pmatrix} 1 & 5 \\ 5 & -6 \\ 2 & 5 \end{pmatrix}$$

Compute AB .

Show work.

☐ Need more space? Continued here: _____

Solution:

$$\begin{pmatrix} 7 & 2 & 0 \\ 1 & 2 & 3 \\ 2 & -3 & -4 \end{pmatrix} \begin{pmatrix} 1 & 5 \\ 5 & -6 \\ 2 & 5 \end{pmatrix} = \begin{pmatrix} 7+10+0 & 35-12+0 \\ 1+10+6 & 5-12+15 \\ 2-15-8 & 10+18-20 \end{pmatrix}$$

$$\begin{pmatrix} 7 & 2 & 0 \\ 1 & 2 & 3 \\ 2 & -3 & -4 \end{pmatrix} \begin{pmatrix} 1 & 5 \\ 5 & -6 \\ 2 & 5 \end{pmatrix} = \begin{pmatrix} 17 & 23 \\ 17 & 8 \\ -21 & 8 \end{pmatrix}$$

3. (7%) Consider the set of vertices $S = \{[2,2,0], [0,1,0], [0,2,2]\}$. You know that the vectors $\{[1,0,0], [0,1,0], [0,0,1]\}$ (standard generators) span $\mathbb{R}^3$. Show that the vectors in S also span $\mathbb{R}^3$.

Show work.

☐ Need more space? Continued here: _____

Solution: Since we know that the three standard generators span $\mathbb{R}^3$ we only need to show that we can reach/create the standard generators using these vectors in S , as that would mean we can reach / span everything that the standard generators span.

$$[1,0,0] = \alpha_1[2,2,0] + \alpha_2[0,1,0] + \alpha_3[0,2,2]$$

$$\alpha_1 = 0.5, \alpha_2 = -1, \alpha_3 = 0$$

$$[0,1,0] = \alpha_1[2,2,0] + \alpha_2[0,1,0] + \alpha_3[0,2,2]$$

$$\alpha_1 = 0, \alpha_2 = 1, \alpha_3 = 0$$

$$[0,0,1] = \alpha_1[2,2,0] + \alpha_2[0,1,0] + \alpha_3[0,2,2]$$

$$\alpha_1 = 0, \alpha_2 = -1, \alpha_3 = 0.5$$

Both a more formal (multiplying out and solving for the alphas) and a faster way (understanding which vector in S has to have a coefficient of zero because it is the only one affecting one of the axes and then solving for the remaining alphas) is possible.

It was demonstrated that these three vectors in S can be used to create the standard generators and, thus, they also span $\mathbb{R}^3$.

4. (7%) Consider the following system of linear equations:

$$1x_1 + 2x_2 + 2x_3 = 1$$

$$3x_1 + 4x_2 + 1x_3 = 11$$

$$2x_1 + 6x_2 + 8x_3 = -4$$

Write it as matrix-vector equation $Ax = b$. Use Gaussian Elimination (or elementary row operations) to find the echelon form matrix of matrix A .

Show work.

☐

Need more space? Continued here: _____

Solution: First we reformulate the system of linear equations as a matrix-vector equation

$$Ax = b$$

$$\begin{bmatrix} 1 & 2 & 2 \\ 3 & 4 & 1 \\ 2 & 6 & 8 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 1 \\ 11 \\ -4 \end{bmatrix}$$

We now use Gaussian Elimination (or elementary row operations) to find the echelon form matrix of matrix A , where U is the echelon form matrix we are looking for:

$$MA = U$$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 2 \\ 3 & 4 & 1 \\ 2 & 6 & 8 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 2 \\ 3 & 4 & 1 \\ 2 & 6 & 8 \end{bmatrix}$$

$$R_2 - 3R_1$$

$$\begin{bmatrix} 1 & 0 & 0 \\ -3 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 2 \\ 3 & 4 & 1 \\ 2 & 6 & 8 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 2 \\ 0 & -2 & -5 \\ 2 & 6 & 8 \end{bmatrix}$$

$$R_3 - 2R_1$$

$$\begin{bmatrix} 1 & 0 & 0 \\ -3 & 1 & 0 \\ -2 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 2 \\ 3 & 4 & 1 \\ 2 & 6 & 8 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 2 \\ 0 & -2 & -5 \\ 0 & 2 & 4 \end{bmatrix}$$

$$Coolumn1done!Column2 : R_3 + R_2$$

$$\begin{bmatrix} 1 & 0 & 0 \\ -3 & 1 & 0 \\ -5 & 1 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 & 2 \\ 3 & 4 & 1 \\ 2 & 6 & 8 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 2 \\ 0 & -2 & -5 \\ 0 & 0 & -1 \end{bmatrix}$$

Matrix U (on the right) is now in echelon form.

5. (7%) Consider the following matrix:

$$A = \begin{bmatrix} -1 & -1 & 1 \\ 1 & 3 & 3 \\ -1 & -1 & 5 \\ 1 & 3 & 7 \end{bmatrix}$$

Give the QR-factorization of the matrix A.

Show work.

☐ Need more space? Continued here: _____

Solution:

$$Q = \frac{1}{2} \begin{bmatrix} -1 & 1 & -1 \\ 1 & 1 & -1 \\ -1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

.

$$R = \begin{bmatrix} 2 & 4 & 2 \\ 0 & 2 & 8 \\ 0 & 0 & 4 \end{bmatrix}$$

6. (7%) Consider the following linear system:

$$Ax = b \tag{1}$$

Given the compact singular value decomposition of $A = U\Sigma V^T$ and the vector b :

$$U = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 0 & 0 \end{bmatrix}, \Sigma = \begin{bmatrix} 2 & 0 \\ 0 & 1 \end{bmatrix}, V^T = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, b = \begin{bmatrix} 1 \\ 2 \\ 0 \end{bmatrix}$$

Solve the linear system (1) by giving $\mathbf{x}$ and reconstruct the matrix $\mathbf{A}$.

Show work.

☐ Need more space? Continued here: _____

Solution:

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 2 \\ 0 & 0 \end{bmatrix}$$

$$x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Part 2

7. Remember that a voting dict is a dictionary which maps senator's last names to their voting record (a list of 1, 0 and -1). A short example is shown in the code snippet below.

```
1 voting_dict = {
2     "Klein": [-1, 0, 1, ...],
3     "Fox-Epstein": [-1, -1, -1, ...],
4     "Ravella": [0, 0, 1, ...],
5     ...
6 }
7
8 def function1(sen_a, sen_b, voting_dict):
9     votes_a = voting_dict[sen_a]
10    votes_b = voting_dict[sen_b]
11    res = 0
12    for i in range(len(votes_a)):
13        vote_a = votes_a[i]
14        vote_b = votes_b[i]
15        res += vote_a * vote_b
16    return res
17
18
19 def function2(voting_dict):
20     worst_pair = None
21     worst_score = float("inf")
22     for sen_a in voting_dict:
23         for sen_b in voting_dict:
24             score = function1(sen_a, sen_b, voting_dict)
25             if score < worst_score:
26                 worst_score = score
27                 worst_pair = (sen_a, sen_b)
28     return worst_pair
```

- (a) (8%) Describe what the function `function1` defined above is computing. Make sure to explain the meaning of the output, not only the arithmetic operations being performed. What does it mean to have an output that is high?

☐ Need more space? Continued here: _____

Solution: The function is computing the dot product between the voting records for a senator a and a senator b. The higher the output is, the more similar are the policy views of the two senators.

- (b) (7%) Describe what the function `function2` defined above is computing. What is the function outputting in terms of its input?

☐ Need more space? Continued here: _____

Solution: The function goes through every pair of voting records in the given voting dict and computes how similar they are using the function `function1`. It then returns the pair of senators that have the most different voting record.

8. (15%) Assume you have a row-list, meaning a list of vectors, representing a matrix. Describe what the function `mysteriousFunction` defined below is computing using the row-list as input. Make sure to explain the meaning of the steps and the output, not only the arithmetic operations being performed. What does the output represent?

```
1 from vec import Vec
2 from mat import Mat
3 import GF2
4
5 def mysteriousFunction(rowlist):
6     """Given a list of vectors, do something mysterious with them.
7     """
8     col_label_list = sorted(rowlist[0].D, key=repr)
9     rows_left = set(range(len(rowlist)))
10    new_rowlist = []
11    for c in col_label_list:
12        rows_with_nonzero = [r for r in rows_left if rowlist[r][c] != 0]
13        if rows_with_nonzero != []:
14            pivot = rows_with_nonzero[0]
15            rows_left.remove(pivot)
16            new_rowlist.append(rowlist[pivot])
17            for r in rows_with_nonzero[1:]:
18                rowlist[r] -= (rowlist[r][c]/rowlist[pivot][c])*rowlist[pivot]
19    return new_rowlist
```

☐ Need more space? Continued here: _____

Solution: The function takes as an input a list of vectors representing a matrix and is used to obtain a new rowlist containing the non-zero rows of the corresponding echelon form matrix (as these form a basis for the row space of the original matrix).

The steps include:

1. Preprocessing (getting the column labels, create an empty new rowlist to later fill, have a list of the rows not yet used)
2. Iterate over each column to get the (non-zero) vectors echelon form matrix
3. Find all rows that have nonzero entries in the column
4. If there are rows with nonzero entries, select the first as the pivot element to then use to put the element in the remaining rows to zero.
5. Take the row corresponding to the pivot element and add it to the new rowlist and remove the corresponding index from the list of remaining rows
6. For all rows for which the element at the current column is still non-zero, use a multiple of the pivot row to put that element to zero
7. After all columns were iterated over, return the new rowlist of non-zero vectors corresponding to the rows of the echelon form matrix of the input matrix (excluding zero vectors, which are not part of the new row list)

9. (15%) **Complete** the following function name `norm_F` that compute the Frobenius norm of a given matrix. Be sure to provide an optimized code, the **solution computing the norm using two embedded loops (i.e `for i in range(...)` `for j in range(...)`) will not have the full grade.**

```
1 from matutil import mat2coldict
2 from math import sqrt
3
4 def norm_F(A):
5     """
6     Input:
7         A: Mat object, a matrix
8     Output:
9         ||A||_F: scalar, Froebinius norm of A
10    Example:
11        >>> A = Mat(({0, 1, 2, 3, 4}, {0, 1, 2}), {(0, 0): 1.19100357,
12            (0, 1): 1.18626107, (0, 2): -0.15051037, (1, 0): 0.07894529,
13            (1, 1): 0.64890449, (1, 2): 0.34588649, (2, 0): -0.03392502,
14            (2, 1): -1.10517013, (2, 2): 0.21532331, (3, 0): 1.50999923,
15            (3, 1): -1.92819375, (3, 2): -1.0304994, (4, 0): 0.24605995,
16            (4, 1): 0.32047944, (4, 2): 0.28975207})
17        >>> normA = norm_F(A)
18        >>> normA == 3.459969330856746
19    """
20    pass
```

Solution:

```
1 def norm_F(A):
2     """
3     Input:
4         A: Mat object, a matrix
5     Output:
6         ||A||_F: scalar, Froebinius norm of A
7     Example:
8         >>> A = Mat(({0, 1, 2, 3, 4}, {0, 1, 2}), {(0, 0): 1.19100357,
9             (0, 1): 1.18626107, (0, 2): -0.15051037, (1, 0): 0.07894529,
10             (1, 1): 0.64890449, (1, 2): 0.34588649, (2, 0): -0.03392502,
11             (2, 1): -1.10517013, (2, 2): 0.21532331, (3, 0): 1.50999923,
12             (3, 1): -1.92819375, (3, 2): -1.0304994, (4, 0): 0.24605995,
13             (4, 1): 0.32047944, (4, 2): 0.28975207})
14         >>> normA = norm_F(A)
15         >>> normA == 3.459969330856746
16    """
17    A = mat2coldict(A)
18    norm = 0
19    for v in range(len(A)):
20        norm_v = A[v]*A[v]
21        norm = norm + norm_v
22    return sqrt(norm)
```

10. (15%) **Complete** the following function `best_k_approximation` to output the matrix A_k being the bestk approximation obtained from the singular value decomposition (SVD) of the matrix A and the Frobenius norm of $A - A_k$. To perform the svd of A you can use the method "factor" from the package svd which for a given matrix A output the matrix U, Σ , V, the function `norm_F` can be use to compute the Frobenius norm of a matrix.

```
1 import svd
2 from matutil import rowdict2mat, coldict2mat
3 import itertools
4
5 def Best_k_approximation(A,k):
6     """
7     Input:
8         A: Mat object, matrix mxn
9         k: integer, number of selected singular vectors,
10    Output:
11        A_k: Mat object, the matrix mxn best-k approximation of A.
12        ||A-A_k||: Frobenius norm of A-A_k
13    Example:
14        >>> A = Mat(({0, 1, 2}, {0, 1, 2}),
15        {(0, 0): 1.19100357, (0, 1): 1.18626107, (0, 2): -0.15051037,
16        (1, 0): 0.07894529, (1, 1): 0.64890449, (1, 2): 0.34588649,
17        (2, 0): -0.03392502, (2, 1): -1.10517013, (2, 2): 0.21532331})
18        >>> A_k, norm_k = Best_k_approximation(A,2)
19        >>> A_k == Mat(({0, 1, 2}, {0, 1, 2}),
20        {(0, 0): 1.1910121958628588, (0, 1): 1.186261189834952, (0, 2): -0.15044113585497326,
21        (1, 0): 0.03516572131849438, (1, 1): 0.6482962815680587, (1, 2):
22        -0.005503400717070249,
23        (2, 0): -0.059588714960482736, (2, 1): -1.1055266633359795, (2, 2):
24        0.009337658721302912})
25        >>> norm_k == 0.4104640320480346
26    """
27    pass
```

Solution:

```
1 def Best_k_approximation(A,k):
2     """
3     Input:
4         A: Mat object, matrix mxn
5         k: integer, number of selected singular vectors,
6    Output:
7        A_k: Mat object, the matrix mxn best-k approximation of A.
8        ||A-A_k||: Frobenius norm of A-A_k
9    Example:
10        >>> A = Mat(({0, 1, 2}, {0, 1, 2}),
11        {(0, 0): 1.19100357, (0, 1): 1.18626107, (0, 2): -0.15051037,
12        (1, 0): 0.07894529, (1, 1): 0.64890449, (1, 2): 0.34588649,
13        (2, 0): -0.03392502, (2, 1): -1.10517013, (2, 2): 0.21532331})
14        >>> A_k, norm_k = Best_k_approximation(A,2)
15        >>> A_k == Mat(({0, 1, 2}, {0, 1, 2}),
16        {(0, 0): 1.1910121958628588, (0, 1): 1.186261189834952, (0, 2):
17        -0.15044113585497326,
18        (1, 0): 0.03516572131849438, (1, 1): 0.6482962815680587, (1, 2):
19        -0.005503400717070249,
20        (2, 0): -0.059588714960482736, (2, 1): -1.1055266633359795, (2, 2):
21        0.009337658721302912})
22        >>> norm_k == 0.4104640320480346
23    """
24    U, Sigma, V = svd.factor(A)
25    U_k = coldict2mat(dict(itertools.islice(mat2coldict(U).items(), k)))
26    V_k = coldict2mat(dict(itertools.islice(mat2coldict(V).items(), k)))
27    Sigma_k = rowdict2mat(dict(itertools.islice(mat2coldict(Sigma).items(), k)))
28    Sigma_k = rowdict2mat(dict(itertools.islice(mat2coldict(Sigma_k).items(), k)))
29    A_k = U_k*(Sigma_k*V_k.transpose())
30    return A_k, norm_F(A-A_k)
```

Part 3

11. (10%) Let V be a set of linearly independent vectors $V = \{v_1, v_2, \dots, v_n\}$. Show that adding a new vector u that is in the span of $v_1, v_2, \dots, v_n$ to the set V does not affect the span, meaning any vector in $\text{Span}(V \cup \{u\})$ is also in $\text{Span}(V)$.

Solution: To show: every vector in $\text{Span}(V)$ is also in $\text{Span}(V \cup \{u\})$

The span of the vectors $v_1, v_2, \dots, v_n, u$ is the set of all linear combinations of these vectors. A vector $v \in \text{Span}(V \cup \{u\})$ can be expressed as:

$$v = \alpha_1 \cdot v_1 + \alpha_2 \cdot v_2 + \dots + \alpha_n \cdot v_n + \alpha_{n+1} \cdot u$$

It is stated that vector u is in $\text{Span}(V)$, meaning there is a linear combination that results in that vector.

$$u = \beta_1 \cdot v_1 + \beta_2 \cdot v_2 + \dots + \beta_n \cdot v_n$$

Plugging in the last equation into the equation for a vector v in $\text{Span}(V \cup \{u\})$, we obtain:

$$v = \alpha_1 \cdot v_1 + \alpha_2 \cdot v_2 + \dots + \alpha_n \cdot v_n + \alpha_{n+1} \cdot (\beta_1 \cdot v_1 + \beta_2 \cdot v_2 + \dots + \beta_n \cdot v_n)$$

Multiplying this out and simplifying it, results in:

$$v = (\alpha_1 + \alpha_{n+1} \cdot \beta_1) \cdot v_1 + (\alpha_2 + \alpha_{n+1} \cdot \beta_2) \cdot v_2 + \dots + (\alpha_n + \alpha_{n+1} \cdot \beta_n) \cdot v_n$$

This shows that an arbitrary vector in $\text{Span}(V \cup \{u\})$ can be written as a linear combination of the vectors in set V ($= \{v_1, v_2, \dots, v_n\}$), and is therefore in $\text{Span}(V)$.

(Similar Proof as for Superfluity)

12. (10%) You are given the following incorrect claim and incorrect proof for the Echelon matrix U obtained from Gaussian Elimination for a matrix A (not in Echelon form) and how it can be used for $Ax = b$. Can you find the error(s) in the claim and proof? Point out and explain the error(s) in the claim and the proof. Then correct the claim and proof.

Claim: Since $MA = U$, the solution x for $Ax = b$ is also the solution for $Ux = b$.

Proof: Suppose v is a solution to $Ux = b$, so $Uv = b$.

Multiply both sides by M^{-1} : $M^{-1}(Uv) = M^{-1}b$

Use associativity: $(M^{-1}U)v = M^{-1}b$

Use property of M^{-1} : Since M was initialized as the identity matrix at the beginning of Gaussian Elimination, $M = M^{-1} = I$ and $M^{-1}b = b$. Thus, we get:

$(M^{-1}U)v = b$

We know $MA = U$ so $A = M^{-1}U$

Use $A = M^{-1}U$: $Av = b$

Thus, the solution v for $Uv = b$ is also a solution for $Av = b$. QED

Solution:

Mistake in Claim: If $MA = U$, then x for $Ax = b$ is NOT also the solution for $Ux = b$. We know this as this implies $A = U$ and we know that U is in Echelon form and A is not (description text), so they are not the same matrix.

Corrected Claim: If $MA = U$, then we can rewrite $Ax = b$ first by multiplying by M so that $MAx = Mb$ and then replacing MA with U , leading to $Ux = Mb$. Thus, **since** $MA = U$, **the solution** x **for** $Ax = b$ **is also the solution for** $Ux = Mb$.

Mistake in Proof: $M^{-1}b = b$. Matrix M is initialized as the identity matrix but changes during Gaussian Elimination. Matrix A is NOT in Echelon form at the beginning of Gaussian Elimination (see question text), so we know M does not remain the identity matrix. We know it is invertible, but it will not remain the identity matrix. Thus, after Gaussian Elimination, neither M nor M^{-1} are the identity matrix and $M^{-1}b$ cannot be simplified to b .

Corrected Proof: Suppose v is a solution to $Ux = b$, so $Uv = Mb$.

Multiply both sides by M^{-1} : $M^{-1}(Uv) = M^{-1}Mb$

Use associativity: $(M^{-1}U)v = M^{-1}Mb$

Since $M^{-1}M$ is the identity matrix

$(M^{-1}U)v = b$

We know $MA = U$ so $A = M^{-1}U$

Use $A = M^{-1}U$: $Av = b$

Thus, the solution v for $Uv = Mb$ is also a solution for $Av = b$. QED

13. (10%) Let be $A \in \mathbb{R}^{m \times n}$ a matrix with a compact singular value decomposition, **prove** that $A^T A$ is **diagonalizable**. Be sure to summon the correct **matrix properties, definitions and theorems** to show step by steps your proof.

Solution:

$$A^T A = (U \Sigma V^T)^T (U \Sigma V^T) \quad (2)$$

$$= V \Sigma (U^T U) \Sigma V^T \quad (3)$$

$(U^T U) = I_m$ because U is a **square and column-orthogonal matrix**, so U^T is the inverse matrix of U.

$$A^T A = V \Sigma \Sigma V^T \quad (4)$$

$$= V \Sigma^2 V^T \quad (5)$$

V is a **square column orthogonal matrix**, by substituting $S = V$, $S^{-1} = V^T$ and $\Lambda = \Sigma^2$, we have

$$A^T A = S \Lambda S^{-1} \quad (6)$$

We have been able to express $A^T A$ a **square matrix** has matrix multiplication between S and Λ . We can re-express (6) as:

$$A^T A v_i = \lambda_i v_i, \quad \forall i \in 1, \dots, n \quad (7)$$

where λ_i is the diagonal element of Λ and v_i the column vectors of S .

By definition, λ and v are the **eigenvalues and eigenvectors** of $A^T A$, and v_1 to v_n are **nonzero linearly independent eigenvectors**, then $A^T A$ is **diagonalizable**.